



Universidade Federal de Itajubá - UNIFEI - Campus Itabira
Curso de Engenharia da Computação - ECO
Computação Gráfica e Processamento Digital de Imagens
Prof. Dr. André Ribeiro de Brito

Fabício Rickelmer Souza Duarte

Instituto de Ciências Tecnológicas, Universidade Federal de Itajubá, Itabira, Minas Gerais, Brazil

d2023012884@unifei.edu.br

R.A: 2023012884

Abstract—Esse artigo tem como fundamento a implementação de um algoritmo genético utilizando mutações, cruzamentos e torneios para a procura de horários ideais e preços baseado em um arquivo de texto para o estudo da matéria de Inteligencia Artificial ministrada na UNIFEI pelo professor Sandro Carvalho Izidoro.

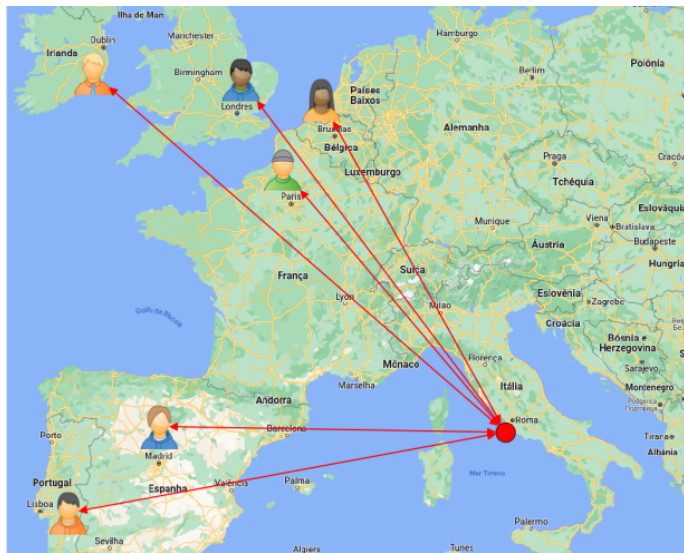
Keywords—Genético, algoritmo, viagens, torneio, cruzamento, mutação, população.

I. INTRODUÇÃO

A. Problema Escolhido

Foi escolhido a atividade 1 do trabalho passado, que era sobre implementar um Algoritmo Genético para otimizar os valores das passagens e os horários dos voos, procurando o menor tempo de espera e custo monetário.

Fig. 1. Pesquisadores e suas respectivas cidades com destino a Roma.



Fonte: Autor Sandro Izidoro (2026).

Os dados que serviram de espaço de busca foram retirados do "flights.txt", que possuía lugares de saída e chegada, hora de saída e chegada; Passados junto ao documento do trabalho.

B. Funcionamento do GA

É um algoritmo bio inspirado que visa encontrar os melhores tipo de solução de um problema dado baseado em variáveis determinadas. Ele não leva em conta todas as possibilidades do cenário biológico inspirado, existe um certo grau de abstração para ele poder ser levado a um código computacional.

1) *Variáveis*: Esse são os parâmetros principais do GA, que vão garantir o seu funcionamento de forma eficaz.

- int tam_Populacao = 15;
O espaço de busca não era tão grande então não foi utilizado um tamanho de população alto.
- int geracoes = 100;
Número maior pra procurar todas as possibilidades possíveis.
- float prob_Cruzamento = 0.80;
O cruzamento alto garante herança de bons padrões para o algoritmo.
- float prob_Mutacao = 0.05;
Baixo pra não ocorrer a geração bem próxima da aleatoriedade.
- int tam_torneio = 2;
Nesse tipo de situação o tamanho 2 é o mais certo pra garantir que genes dominantes não tomem conta do algoritmo.

II. DESENVOLVIMENTO

A. Bibliotecas Utilizadas

1) C++:

- <iostream>
Responsável pela entrada e saída padrão do sistema. Utilizada no código para exibir os relatórios de progresso, o resumo do melhor indivíduo encontrado, além de mensagens de sucesso e alertas de erro diretamente no terminal de execução.
- <fstream>
Biblioteca responsável pela manipulação de arquivos do disco rígido. Foi utilizada através do comando ifstream para abrir e ler o banco de dados inicial (flights.txt), e do comando ofstream para criar, nomear e gravar os dados exportados (como os relatórios de texto .txt e a planilha de dados estatísticos .csv).

- `<sstream>`
Ferramenta de conversão e processamento estruturado de strings. Na função de verificação e leitura do arquivo de voos, transforma cada linha de texto em um fluxo contínuo de dados (String Stream), permitindo isolar as informações delimitadas por vírgulas (origem, destino, horários e preço) para conversão numérica.
- `<vector>`
Fornece a estrutura de dados de listas dinâmicas, que expandem de tamanho automaticamente na memória. Atua como a espinha dorsal de armazenamento do programa, sendo utilizada para guardar o espaço de busca inicial, agrupar os voos selecionados para compor o cromossomo, gerenciar a população atual a cada geração e acumular o histórico estatístico global.
- `<random>`
Disponibiliza motores modernos de geração de números pseudoaleatórios. Implementada através do motor Mersenne Twister (mt19937) em conjunto com a leitura do hardware (random_device). Constitui o núcleo das operações estocásticas do algoritmo, determinando o sorteio de rotas na criação da população, definindo os vencedores nos torneios de seleção e controlando as taxas de probabilidade nos gatilhos de cruzamento e mutação.
- `<algorithm>`
Biblioteca padrão que fornece algoritmos utilitários otimizados, acionada estruturalmente pelo sistema para a indexação, contagem e manipulação interna de elementos contidos nos vetores iteráveis do projeto.

2) Python:

- pandas
Biblioteca de alta performance com foco em análise e manipulação de dados em formato tabular. No script de renderização, atua como a interface de leitura, importando o arquivo .csv exportado pelo C++ e convertendo os dados bidimensionais em um DataFrame, o que possibilita a indexação direta das métricas evolutivas.
- matplotlib.pyplot
Biblioteca primária para a renderização de gráficos estáticos e plotagem de precisão. Funciona como o motor de processamento visual do projeto, recebendo os vetores numéricos estruturados para desenhar as curvas de convergência em eixos cartesianos, aplicar as máscaras de desvio padrão e, por fim, exportar o ambiente de plotagem para arquivos de imagem de alta resolução na pasta designada.

B. Estruturas

1) C++: Estruturas para a implementação do GA

Listing 1. Estruturas do GA em C++

```
1 // Estrutura para receber as variaveis do .
  txt
2 struct Voo{
3     string origem;
4     string destino;
5     string hora_saida;
```

```
6     string hora_chegada;
7     double preco;
8 };
9
10 // Estrutura para a geracao do individuo
11 struct Indivduo {
12     vector<Voo> cromossomo;
13     double fitness;
14 };
15
16 // Estrutura para as estatisticas
17 struct Estatisticas {
18     int geracao;
19     double melhor_fitness;
20     double pior_fitness;
21     double media_fitness;
22     double variancia;
23     double desvio_padrao;
24 };
```

C. Funções

- `verifica_texto`
Responsável por abrir e ler o arquivo de texto contendo os dados dos voos disponíveis, processar cada linha e armazenar as informações estruturadas dentro do vetor que representa o espaço de busca geral do sistema.
- `sortear_voo_por_origem`
Filtra todos os voos do catálogo que possuem a cidade de origem especificada e realiza o sorteio aleatório de um deles utilizando o gerador pseudoaleatório informado por parâmetro.
- `converte_tempo`
Realiza a conversão de um horário representado em formato de string contendo horas e minutos para um valor numérico inteiro equivalente em minutos totais, simplificando as operações matemáticas de tempo do programa.
- `calculo_fitness`
Avalia e atribui a nota de aptidão de um indivíduo com base no somatório do preço de todas as passagens e no tempo total de espera acumulado nos voos de seu cromossomo.
- `gerar_populacao`
Instancia o conjunto inicial de soluções do algoritmo baseado no tamanho populacional desejado, sorteando uma rota inicial válida para cada indivíduo e calculando suas respectivas notas.
- `torneio`
Aplica a técnica de seleção por torneio para escolher um indivíduo da população atual, comparando a aptidão de um subgrupo sorteado para definir quem possui o melhor fitness.
- `mutacao`
Altera de forma estocástica o material genético de um indivíduo com base em uma probabilidade definida, sorteando um novo voo para uma determinada cidade de origem com o intuito de manter a diversidade.
- `cruzamento`
Combina as características de dois indivíduos pais para

a geração de dois novos indivíduos filhos através de um sorteio binário que define a origem de cada gene do cromossomo.

- **o_melhor**
Percorre todos os membros da população fornecida para identificar e retornar o indivíduo que obteve o menor valor de fitness, o qual representa a solução mais otimizada daquela etapa.
- **imprime_exporta**
Realiza a exibição da listagem completa dos indivíduos da população inicial no terminal e efetua a gravação idêntica dos dados em um arquivo de texto específico.
- **imprime_exporta_final**
Apresenta os dados de todos os indivíduos que compõem a população da última geração no terminal e salva o relatório correspondente em um arquivo de texto estruturado.
- **imprime_individuo**
Exibe o arranjo de colchetes com os voos e a nota do melhor indivíduo absoluto no terminal e grava essas informações resumidas em um arquivo de texto na pasta correspondente.
- **calcular_estatisticas**
Processa os dados de desempenho de uma determinada geração para extrair os valores do melhor fitness, pior fitness, média geral da população, variância e desvio padrão.
- **exportar_dados_csv**
Recebe o vetor histórico contendo as métricas de todas as rodadas evolutivas e grava os valores estruturados em uma planilha com formato de valores separados por vírgula para possibilitar a plotagem externa dos gráficos.

D. Resultados Gerados

1) Arquivos de Texto:

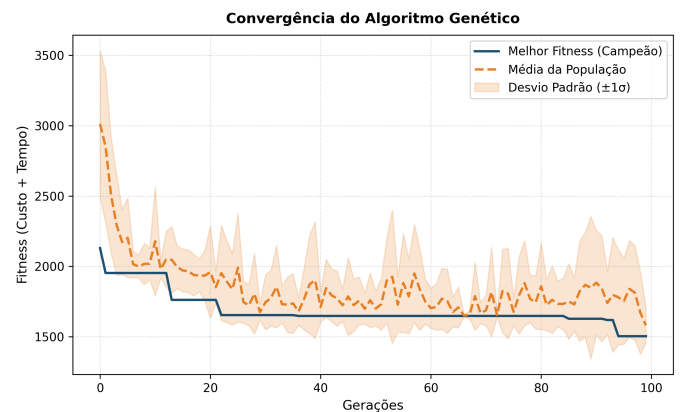
- **Populacao_Gerada_Inicial.txt**
Este arquivo armazena o estado bruto da primeira geração do algoritmo. Ele lista todos os indivíduos criados aleatoriamente no início da execução, detalhando os voos de cada cromossomo e suas respectivas notas de aptidão (fitness). Serve como um ponto de referência para avaliar o nível de dispersão e a qualidade das rotas antes de qualquer processo de otimização.
- **Populacao_Gerada_Final.txt**
Contem a listagem completa de todos os indivíduos pertencentes a última geração do laço evolutivo. O arquivo documenta o estado da população após passar por todos os ciclos de seleção, cruzamento e mutação, permitindo observar fisicamente a convergência dos genes e a similaridade entre os indivíduos restantes.
- **Melhor Indivíduo.txt**
Funciona como o relatório final e direto do sistema, isolando e exibindo unicamente a solução mais otimizada encontrada em toda a execução. O arquivo detalha o fitocrenes do campeão absoluto e a sequência exata de voos, incluindo as informações de origem, destino, horários e preço, que resultou no menor custo e tempo de espera.

• Graficos_GA.csv

É um arquivo de valores separados por vírgula que atua como o banco de dados histórico da evolução. Ele registra, linha por linha, as estatísticas precisas de cada geração processada, armazenando o identificador da geração, o melhor e pior fitness, a média populacional, a variância e o desvio padrão. Este arquivo é a base de dados consumida pelo script externo para a renderização das curvas de convergência.

2) *Estatísticas e Gráficos Gerados:* No gráfico da figura 2 é demonstrada a eficácia do algoritmo que foi rodado com 100 gerações e atinge o ótimo global em torno de 1500. A linha pontilhada mostra a variação de melhores indivíduos encontrados em cada geração.

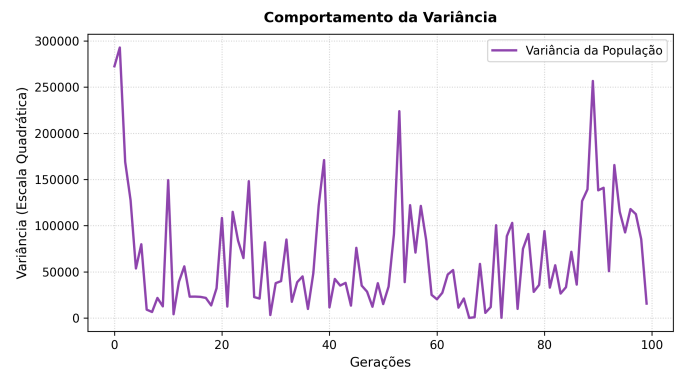
Fig. 2. Convergência dos Valores



Fonte: Autoria Própria (2026).

No gráfico da figura 3 é demonstrado a variação genética, através da mutação inserida no algoritmo.

Fig. 3. Variância Gerada



Fonte: Autoria Própria (2026).

III. CONCLUSÃO

Durante o trabalho foi possível verificar o funcionamento do algoritmo genético mudando suas variáveis para observar o seu peso e como deveria ser ajustados conforme o problema.

Foi possível verificar que esse tipo de implementação pode ser usada em diferentes tipos de problemas e com adequações pode ser muito útil, mas deve ser usada com muito estudo de caso pois pode facilmente virar um algoritmo de aleatoriedade sem eficiência.

Foi utilizado um código em python para demonstração visual dos resultados e melhor compreensão das estatísticas com o passar das gerações.

REFERENCES

- [1] Cppreference Contributors. **C++ Reference**. 2026. Disponível em: <https://en.cppreference.com/>. Acesso em: 12 jun. 2026.
- [2] cplusplus.com. **Reference - C++ Information**. 2026. Disponível em: <https://cplusplus.com/>. Acesso em: 12 jun. 2026.
- [3] The pandas development team. **pandas - Python Data Analysis Library**. 2026. Disponível em: <https://pandas.pydata.org/>. Acesso em: 12 jun. 2026.
- [4] Matplotlib Development Team. **Matplotlib: Visualization with Python**. 2026. Disponível em: <https://matplotlib.org/>. Acesso em: 12 jun. 2026.
- [5] Disciplina de Inteligência Artificial (ECO122). **Atividade Algoritmos Evolucionários**. Material de avaliação disponibilizado via SIGAA. 2026. Atividade baseada no trabalho dos professores Jones Granatyr e Guilherme Matos Passrini.